mangucletus / AI-Engineering-Course

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights

Settings

AI-Engineering-Course / week-5 / ophelia-week5-day4.md

opheliaayamga246810-arch Update ophelia-week5-day4.md 25dd054 · now

Ophelia: Week 5, Day 4 - Pros & Cons of RAG, Evaluation

Pros of RAG

1. Rapid time to Market
2. Give an LLM vast expansive expertise
3. Save on Context length
4. Avoid distracting with irrelevant Context

Cons of RAG

1. It's a hack
2. It's empirical aka trial & error
3. Unexpected 'I don't know'

4. Great idea can have poor results.

Case-in-point: chunking strategy can have a significant impact on performance

THE SIMPLE TECHNIQUE TO GET PERFORMANCE FROM YOUR RAG PIPELINE

Evaluation are Everything

- Create a Test set, Example questions set with the right Context identified and reference answers provided.
- Measure Retrieval - MRR (Mean Reciprocal Rank) nDCG (Normalized Discounted Cumulative Gain) Recall@K and Precision@K
- Measure Answers - e.g. use LLM-as-a-judge to score provided answers against criteria like accuracy, Completeness and

- MRR = Average inverse rank of first hit; 1 if the first chunk always has relevant context.
- nDCG = Did relevant chunks get ranked higher up
- Recall@K = Proportion of Test where Relevant Context was in the top K chunks or If you have multiple keywords to look for, Keyword coverage is similar recall metric
- Precision@K = Proportion of the top K chunks that are relevant.

Code

```
from evaluation import test

test = test.load_tests()
len(tests)

example = test[0]
print(example.question)
```

```
    print(example.category)
    print(example.reference_answer)
    print(example.keywords)
```